

Contents

Toxico Learning - Application Documentation	1
Table of Contents	1
1. System Overview	2
2. User Roles	2
3. Student Registration Flow	4
3.1 Entry points	4
3.2 Authentication primitives (src/lib/auth.svelte.ts, src/lib/firebase.ts)	4
3.3 Step-by-step: new student signup	4
3.4 Step-by-step: returning student login (LoginForm.svelte)	5
4. Student Classroom Interaction and Requirements	5
4.1 Browsing classes	5
4.2 Class detail layout (/classes/[id] -> StudentDashboard.svelte)	6
4.3 Check-in flow (required to unlock a lecture)	6
4.4 Lecture content (LectureDetailPanel.svelte + MaterialRenderer)	6
4.5 Quizzes (/quiz/[id]/take + QuizTaker.svelte)	7
4.6 Assignments (AssignmentDetailPanel.svelte)	8
4.7 What is required of a student (summary)	8
5. Admin Classroom Management	8
5.1 Dashboard entry (/dashboard)	8
5.2 Single class editor (/dashboard/[classId] -> ClassEditor.svelte)	9
6. Lecture Creation (including Google Meet)	9
6.1 Lecture editor (LectureEditor.svelte)	9
6.2 Google Meet integration	10
7. Admin Checking Attendance	11
8. Admin Managing Students (and Teachers)	11
8.1 Per-class enrolment (/classId/students -> EnrolStudents.svelte)	11
8.2 Global student management (/dashboard/students -> ManageStudents.svelte)	12
8.3 Google Sheets import (GoogleSheetsImport.svelte)	12
8.4 Teacher management (/dashboard/teachers -> ManageTeachers.svelte)	12
8.5 Authorization summary	12
9. Data Model Reference	12
10. Appendix: Routes and Cloud Functions	13

Toxico Learning - Application Documentation

Project: toxico-learning-svelte (SvelteKit 2 + Svelte 5 + Firebase) Firestore project:
rama-toxico-edu Date: September 2026

Table of Contents

1. System Overview
2. User Roles
3. Student Registration Flow
4. Student Classroom Interaction and Requirements
5. Admin Classroom Management

6. Lecture Creation (including Google Meet)
 7. Admin Checking Attendance
 8. Admin Managing Students (and Teachers)
 9. Data Model Reference
 10. Appendix: Routes and Cloud Functions
-

1. System Overview

Toxico Learning is a blended-learning web application for toxicology electives (medical students and residents). It has two portals:

Portal	Entry route	Audience
Student portal	/ (login) -> /classes	Students, Residents
Admin / Teacher dashboard	/login (staff) -> /dashboard	Teachers, Admins
Public signup	/signup	New students (Google sign-up)
Quiz authoring	/quiz	Teachers, Admins

Tech stack:

- Frontend: SvelteKit (file-based routing), Svelte 5 runes (\$state, \$derived, \$effect), Tailwind CSS + DaisyUI, Lucide icons.
- Backend: Firebase Authentication, Cloud Firestore (with persistent multi-tab cache), Cloud Storage, Cloud Functions (us-central1-rama-toxico-edu.cloudfunctions.net), BunnyCDN for uploaded video, Google Meet + Google Calendar APIs for live lectures.

High-level architecture:

Browser (SvelteKit)

- |-- Firebase Auth (Google popup for students/teachers, email+password for admins)
 - |-- Firestore (classes, lectures, assignments, quizzes, users, activities, quizAttempts)
 - |-- Cloud Storage (assessment-forms/*.pdf, assignment submissions, lecture files)
 - `-- Cloud Functions (signUpGoogle, googleSignIn, enrolStudents, submitQuiz, submitAssignment, createMeetSpace, getMeetParticipants, ...)
-

2. User Roles

Role	Years / extra fields	How created	Landing page	Capabilities
student	Year y4, y5, y6	Self-signup via Google (/signup)	/classes	View enrolled classes, check in, complete lectures/quizzes, submit assignments
resident	Year r1, r2, r3 + required department	Self-signup via Google (/signup)	/classes	Same UX as student
teacher	-	Invited by admin (inviteTeacher), then completes Google sign-in	/dashboard	Create/edit lectures and quizzes, create Meet sessions, view attendance
admin	-	Created by email/password (/login, createUsers / updateUserRole)	/dashboard	Everything a teacher can do, plus: create/delete classes, enrol/remove students, manage users, import from Google Sheets, manage templates

Roles are stored both in the Firestore users document and as Firebase Auth custom claims (role), enforced by Firestore security rules and Cloud Functions (verifyAdmin, verifyStaff).

Firestore access summary:

- **classes:** read by any authenticated user; write admin-only.
- **lectures (classes/{id}/lectures):** read authed; write teacher/admin.
- **assignments:** read authed; write admin.
- **quizzes:** read authed; write teacher/admin.
- **quizAttempts:** create/read own only.
- **users/{id}/activities/{lectureId}:** owner-only read; create requires checkedInAt and request.time <= lecture.endTime; update keeps checkedInAt immutable.
- **users:** read owner or staff; write admin.
- **Enrolment changes** go through the enrolStudents Cloud Function (admin-only), which updates both users.enrolledClasses and classes.enrolledStudents with arrayUnion.

3. Student Registration Flow

3.1 Entry points

- / - student login page (LoginForm.svelte). If a profile already exists, redirects to /classes.
- /signup - student signup page (SignupForm.svelte). If profile exists and role is teacher/admin, redirects to /dashboard; otherwise to /classes.
- /login - staff login page (AdminLogin.svelte, email + password). Only role == admin is accepted; anyone else is signed out with an adminOnly error and sent to /dashboard only on success.

3.2 Authentication primitives (src/lib/auth.svelte.ts, src/lib/firebase.ts)

1. Firebase Auth with Google provider (signInWithPopup) for students/teachers; email/password for admins.
2. On auth-state change, the app queries users where authId == uid to build a UserProfile (name, email, role, photoURL, year, uid, docId). If no document exists, the profile is null (a Google user who has not registered yet).
3. refreshProfile() forces getIdToken(true) so fresh custom-claim role values take effect after signup or invitation.

3.3 Step-by-step: new student signup

Stage 1 - Connect Google account (stage = 'connect'):

1. Student clicks Google sign-in -> signInWithPopup(googleProvider).
2. Frontend calls POST googleSignIn with Authorization: Bearer <idToken> (functions/googleSignIn.js):
 - Verifies the token was issued for sign_in_provider == google.com.
 - Looks up users by email or emailLower.
 - No document found -> returns { enrolled: false } -> UI advances to the registration form (stage = 'form').
 - Teacher invite found (role == teacher && !signedUp) -> migrates the invite document to users/{uid}, deletes the old doc, sets custom claim teacher, returns { enrolled: true, role: 'teacher', pendingTeacher: true } -> redirect to /dashboard.
 - Existing user -> updates authId, email, emailLower, displayName, photoURL, signedUp/signedUpAt, sets claim role -> returns { enrolled: true, role }.
 - If enrolled is true the student is sent to /classes (teachers to /dashboard).

Stage 2 - Registration form (stage = 'form', all fields required unless noted):

- firstName, lastName
- gender: male | female
- role: student | resident
- year: y4, y5, y6 for students; r1, r2, r3 for residents (changing role resets an incompatible year)
- hospital
- department - required only when role == resident
- phone
- lineId (optional)

- electiveStart date, electiveEnd date
- assessmentFile: PDF, max 10 MB

Validation on the client:

- Year must match the selected role.
- Buddhist-year guard: a year value > 2100 is rejected with a buddhistYearNote error and blocks submit.
- File must be application/pdf (or .pdf extension) and < 10 MB, otherwise assessmentFormInvalidType / TooLarge errors.

Submit sequence:

1. Upload PDF to Cloud Storage at assessment-forms/{uid}/{uuid}.pdf -> obtain download URL.
2. POST signUpGoogle with all form fields plus assessmentFormUrl/Path/Name.
3. On failure the uploaded file is deleted, the user is signed out, and the error is shown.

Backend validation (functions/signUpGoogle.js):

- Bearer token must be a Google sign-in and include an email.
- VALID_ROLES = [student, resident], VALID_GENDERS = [male, female], year lists per role as above.
- firstName, lastName, gender, role, hospital, phone, year are required; department required for residents; assessmentFormUrl/Path/Name required and Path must start with assessment-forms/.
- electiveStart/End must parse to valid dates.
- Duplicate email (email in [email, emailLower]) -> 409 Email already registered and the newly created Auth user is deleted.
- On success creates users/{uid} with authId, email, emailLower, name, gender, role, year, hospital, department?, phone, lineId?, electiveStart/End, assessmentForm*, enroledClasses: [], signedUp: true, signedUpAt, createdAt and sets the custom claim { role }.

3.4 Step-by-step: returning student login (LoginForm.svelte)

1. signInWithPopup -> POST googleSignIn.
2. Request fails -> sign out and show auth.notEnrolled.
3. pendingTeacher or role == teacher -> refresh token and go to /dashboard.
4. !enrolled -> go to /signup.
5. Otherwise refresh profile and go to /classes.

4. Student Classroom Interaction and Requirements

4.1 Browsing classes

- /classes renders ClassesList.svelte. StudentDashboard loads the student's classes (those whose enroledStudents contains the student's doc id) plus users/{docId}/activities. An empty state shows dashboard.noClassesYet. Clicking a class goes to /classes/{id}.

4.2 Class detail layout (/classes/[id] -> StudentDashboard.svelte)

Three-pane layout (DashboardLayout sidebar + LectureListPanel + detail panel):

- Loads classes/{id}/lectures (id, title, startTime, endTime, materials, materialsOrder).
- Loads assignments: classes/{id}/assignments where assignedStudentIds array-contains docId, sorted by dueDate.
- Breadcrumb: All Classes > ClassName > Lecture | Assignment.

Lecture list (LectureListPanel.svelte):

- Grouping: Today (highlighted card), Upcoming (ascending), Past (descending). Each row shows formatTimeRange, a Meet icon when applicable, and checkedIn / completed badges (emerald when done, yellow when pending with timestamp tooltip). Past lectures that are still incomplete show a red notCheckedInOrCompleted alert.
- Accessibility gate isAccessible(lecture) (uniform for all lectures, with or without Meet): accessible while now is in [startTime - 15 min, endTime].
 - Inaccessible lectures show a lock icon, are disabled and dimmed (opacity-50).
- Assignments section lists each assignment with opens / due MMM D · hh:mm A.

4.3 Check-in flow (required to unlock a lecture)

1. Click a lecture. If its id is already in checkedInIds, it opens directly; otherwise a check-in modal appears.
2. The modal shows the lecture name, live device time (hh:mm:ss A, ticking), start time (ddd, MMM D · hh:mm A), and the rule text (checkInEarlyUntilEnd: check-in open from 15 minutes before start until the lecture ends).
3. canCheckIn rule (uniform for all lectures): start - 15 min <= now <= end. Outside the window the button is disabled with an outsideCheckInWindow message.
4. confirmCheckIn writes setDoc(users/{docId}/activities/{lectureId}, { classId, lectureId, checkedInAt: serverTimestamp() }) and updates the local activities list.

Firestore additionally enforces: activity creation requires checkedInAt != null, matching lectureId, and request.time <= lecture.endTime; checkedInAt is immutable on update.

4.4 Lecture content (LectureDetailPanel.svelte + MaterialRenderer)

Header shows title, date, checkedIn hh:mm A time, completed state, and Meet host (displayName | email).

Material types rendered per item:

Type	Rendering
youtube video	Embedded YouTube player BunnyCDN video resolved via POST getVideoPlaybackUrl { videoId }
link / file text	Link card with open/download Formatted text block

Type	Rendering
quiz	MaterialQuiz: red REQUIRED pill if requiredPostTest; PASSED / FAILED pills with timestamps; otherwise clickToTakeQuiz
meet	Join button + autoCheckInNote (“Participants will be auto checked-in after meeting ends”)

If the lecture has a Meet session, an attendance card shows sessionName, Joined / Left times, and humanized Duration (sourced from the student’s own activities/{lectureId}.meetSession).

The lecture header shows an organized status list: Checked in time (or pending), Completed time (or pending), and - for lectures containing a Meet material - an Attended meeting row. That row is resolved from the lecture document’s cached sessionData.participants[] (matched by user id or email): Attended · hh:mm A when matched, Absent once a conference record exists but the student is not in it, and Not recorded yet before any session data is available.

Mark as Completed - this button is rendered for every lecture. Above it, a Completion requirements card lists each gate: required post-test quizzes (pass/fail, tappable to start), uploaded videos with watch progress (each must be watched past 80%), and - for lectures containing a Meet material - Meet attendance resolved from the lecture’s sessionData (must show Attended). The button stays disabled until all gates pass; otherwise a hint is shown (pleaseWatchVideos / pleaseAttendMeet). Completing writes setDoc(activities/{id}, { classId, lectureId, completedAt: serverTimestamp() }, { merge: true }) and shows a greatJob / markedAsCompleted modal. Video progress itself is tracked per video as a 0-1 fraction in activities/{lectureId}.videos[] (flushed every 2 min and on video end) and shown on each video card as a progress bar.

4.5 Quizzes (/quiz/[id]/take + QuizTaker.svelte)

- Loads quizzes/{id} and prior attempts (quizAttempts where authId == uid && quizId == ..., plus lectureId when in class context).
- Start screen: title, question count, question list, passing score (default 70%), last 5 attempts (score/total (pct%) + date), Start | Retake button.
- Taking: optional shuffleQuestions, progress answered/total + bar; types: multiple-choice (radio), multiple-answer (checkbox), true-false, short-answer (input). Submit is disabled until every question is answered.
- Submit calls POST submitQuiz { quizId, lectureId?, answers }. Note: the function requires quizId + lectureId + answers, so quizzes must normally be taken embedded in a lecture; opening /quiz/.../take directly without a lecture fails.
- Grading: MC/TF by exact match, multi-answer by same-length set match, short-answer by case-insensitive substring. $pct = score/total * 100$, $passed = pct \geq passingScore$ (70 default). Stores quizAttempts/{autoId} and returns { id, score, totalPoints, passed, pct } -> result modal (quizPassed / Failed, percentage, points) -> View Attempts.
- In class context the app picks the first passing attempt (earliest), else the latest attempt; this drives allRequiredPassed.

4.6 Assignments (AssignmentDetailPanel.svelte)

- Loads the student's submission .../assignments/{id}/submissions/{docId} (attachments, submittedAt, updatedAt).
- Windows from opensAt / dueDate: before open -> submissionsOpen notice; after due -> pastDue (upload/save/submit disabled); submittable only when opens <= now <= due.
- For each requiredAttachments[{ id, instruction }]: lists matching attachments (name, uploadedAt, View via getDownloadURL, Remove via deleteObject + save); FileUpload uses uploadBytesResumable to assignments/{aid}/submissions/{docId}/{uuid.ext} with progress %, then auto-saves draft (setDoc merge with studentUserId, updatedAt, attachments[{ name, filePath, attachmentId, uploadedAt }]).
- Footer: Save draft (disabled unless there are attachment changes and no upload in progress) + Submit (calls POST submitAssignment { classId, assignmentId }). The backend verifies the user, compares required vs. provided attachment ids, writes { requirementsMet, missingAttachmentIds, updatedAt, submittedAt }, and returns { requirementsMet, requiredCount, providedCount, missing }. UI shows lastSubmitted / lastSaved and couldNotSave / Upload / Submit errors.

4.7 What is required of a student (summary)

There is no single percentage threshold in code; completion is per lecture:

1. Check in within the allowed window to unlock materials.
2. Pass every required post-test quiz (unlimited retakes; typically 70% to pass) and watch every uploaded video past 80%, then click Mark as Completed (gated by the requirements card).
3. Submit assigned assignments within [opensAt, dueDate] with all required attachments.
4. For Meet lectures, join via the Meet link so a Meet session record exists (used for auto check-in evidence alongside manual check-in).

5. Admin Classroom Management

5.1 Dashboard entry (/dashboard)

+layout.svelte loads dashboardStore.loadClasses() plus loadLecturesForClass(classId); the left TreeView lists classes -> lectures with Breadcrumbs; actions: + New Class, Manage Students.

dashboardStore:

- loadClasses(): getDocs(classes) -> { id, name, code, enroledStudents -> students, classStart, classEnd }.
- loadLecturesForClass(classId) (lazy, once): classes/{id}/lectures -> { title, startTime, endTime, materials, materialsOrder }.
- addClass(): addDoc(classes, { name: 'New class', code: '', createdAt: serverTimestamp }) -> navigates to /dashboard/{id}.
- Optimistic local renameClass / updateLecture / insertLecture / deleteLecture / deleteClass.

/dashboard/+page.svelte (All Classes):

- Cards with name, code, dates, students.length; handleAddClass.
- Course templates section (getDocs(courseTemplates) with lectureCount).
- Shortcuts: Manage Students -> /dashboard/students, Manage Teachers -> /dashboard/teachers.

5.2 Single class editor (/dashboard/[classId] -> ClassEditor.svelte)

1. Class details: edit name, code, classStart/classEnd (date-only DateTimeInput) -> updateDoc(classes/{id}, ...) + store sync. Dirty tracking with beforeNavigate / beforeunload guard.
 2. Student block: shows students.length + three buttons: Enrol Students -> /dashboard/{id}/students, Attendance -> /{id}/attendance, Assess -> /{id}/assessment.
 3. Assignments block: getDocs(classes/{id}/assignments) sorted by dueDate; AssignmentCard list; Add Assignment opens a modal with AssignmentEditor -> addDoc(.../assignments, { title, instructions, opensAt, dueDate, requiredAttachments, assignedStudentIds, createdAt }). Editing an assignment goes to /dashboard/{classId}/assignments/{assignmentId}; submissions at .../submissions.
 4. Lectures block: ScheduleCalendar week view (drag to change startTime/endTime -> updateDoc) + All Lectures list (Meet icon when a meet material exists). Add Lecture or clicking a calendar slot creates a draft -> modal LectureEditor -> dashboardStore.insertLecture. Import from Template -> ImportLecturesFromTemplate. Editing opens LectureEditor with save/delete/dirty/discard handling. Deleting the class calls POST deleteClass { classId } (admin-only function) -> store update + redirect to /dashboard.
-

6. Lecture Creation (including Google Meet)

6.1 Lecture editor (LectureEditor.svelte)

Supported material types: video, file, link, text, quiz, meet.

- Header: title (Input), startTime/endTime (datetime-local, validated start < end, Buddhist-year > 2100 check). Save / Create writes classes/{cid}/lectures/{lid} with { title, startTime, endTime, materials: [{ id, type, title, value, requiredPostTest?, meetingHost? }], materialsOrder }.
- Materials are sorted by materialsOrder; per-material UI state (videoMode, videoId, embedUrl, fileUrl, ...). Dirty tracking covers lecture fields + material id/type/title/value/requiredPostTest + order, with a leave-warning modal (beforeNavigate).
- Add material: new id mat-{randomUUID} + default title per type. quiz opens a QuizPicker modal; meet opens a MeetCreator modal (only one meet per lecture; button disabled when hasMeet; requires a non-empty lecture title); all other types are added immediately and persisted via updateDoc (unless the lecture itself is still new).
- Reorder via drag-and-drop (@dnd-kit/svelte); deleting an uploaded video also calls POST deleteVideo { videoId }. Each row is a MaterialCard (editable title, delete, drag handle).

Per-type editors:

- **MaterialLinkEditor**: single URL input.
- **MaterialTextEditor**: multi-line textarea note.
- **MaterialVideoEditor**: tabs YouTube | Upload. YouTube validates via `getYoutubeVideoId()` with live `youtube.com/embed/{id}` preview. Upload: `POST getVideoUploadUrl { title }` -> `PUT https://video.bunnycdn.com/...` with progress -> select video; or pick an existing library video; playback via `POST getVideoPlaybackUrl`.
- **MaterialFileEditor**: `uploadMaterial { originalName, description, fileData (base64) }` via XHR (10 MB limit; image/*, .ppt, .pptx, .docx, .pdf) or pick an existing file via `FilePicker` -> `getDownloadURL`.
- **MaterialQuizEditor**: shows linked quiz title + Edit Quiz (`/quiz/{id}/edit`) + requiredPostTest checkbox (the post-test gate described in §4.4).
- **MaterialMeetEditor**: read-only Meet URL row with copy + open buttons.
- **QuizPicker**: lists `getDocs(quizzes)` (title, questions.length) -> links quiz into the lecture; shortcut to `/quiz/new`.

6.2 Google Meet integration

Step 1 - Connect Google Meet (`GoogleMeetConnect.svelte`, used on `/settings`):

1. `GET meetOAuthStatus { connected }` with the Firebase ID token.
2. `POST meetOAuthUrl {}` -> `{ url }` -> browser redirects to Google consent. Backend (`meetOAuth.js`) verifies staff role, uses `secret/meetAccount.json` (`client_id/secret`), requests scopes `meetings.space.created + calendar.events + contacts.readonly + directory.readonly + gmail.send`, and builds an auth URL with `redirect_uri = {base}/meetOAuthCallback`, `access_type: offline`, `prompt: consent`, `state = <ID token>`.
3. The callback verifies the state token, exchanges code for tokens, stores `meetOAuth/{uid}` `{ refreshToken }`, and redirects to `{app}/settings?meet=success|error` (banner, then history replaced to `/settings`).

Step 2 - Create a meeting (`MeetCreator.svelte` modal, props `classId`, `title`, `startTime`, `endTime`):

1. Load hosts: users where role in `[teacher, admin]`, current user pre-selected; searchable dropdown plus live Google directory via `POST getPeopleDirectory { query }` (debounced 350 ms, merges registered + directory matches). A `connect_google_meet` response links to `/settings`.
2. Load invitees: `POST getMeetInvitees { classId }` -> `{ name, email }[]` resolved from `enroledStudents` (chunked in queries of 30).
3. Confirm & Invite: `POST createMeetSpace { classId, title, startTime, endTime }`. Backend (`meet.js`): verifies staff -> gets access token from the stored refresh token (throws `connect_google_meet` if missing) -> `POST https://meet.googleapis.com/v2/spaces {}` -> `{ meetingUri, meetingCode, name }` -> creates a Calendar event (`calendars/primary/events?conferenceData { hangoutsMeet, meetingUri }`) -> `{ eventId, htmlLink }`. Frontend receives `meetingUri` + host and persists `{ type: 'meet', value: meetingUri, meetingHost }`.
4. Students see `MaterialMeet.svelte`: Join button + note that participants are auto checked-in after the meeting ends.

7. Admin Checking Attendance

Page: /dashboard/[classId]/attendance -> AttendanceView.svelte (admin/teacher only).

Data loading:

- Class `enroledStudents[]`; lectures filtered to those containing a meet material (plus cached `sessionData`); all users mapped by id; per student `users/{uid}/activities` where `classId == ...` -> { `checkedInAt`, `completedAt` } keyed by `lectureId`.

UI:

- Header counts + Export CSV (`attendance_{classId}.csv`: `ramaId`, `name`, `email`, `checkedIn`, `completed`, `total` + per-lecture check-in/completed times).
- Per-lecture Google Meet card: View Session (cached `sessionData`) or Check Participants (POST `getMeetParticipants { meetingUri, classId, lectureId }`, cached via `updateDoc(lectures/{lid}, { sessionData })`).
- Per-student expandable rows with `checkedIn x/y` (emerald ClockCheck) + `completed x/y` (teal ListChecks) and per-lecture timestamps.
- Participants modal: roster joined to Meet participants by lowercased email -> Attended / Absent pills in a table (Full name | Status | Display name | Enter | Leave | Duration). Refresh forces reload.

Backend (`getMeetParticipants`, `meet.js`, staff-only):

1. Extract meeting code from `meetingUri.split('/').pop()` -> GET `conferenceRecords?filter=space.meeting` -> if none, { `conferenceRecord`: null, `participants`: [] }.
2. GET `{record}/participants` -> for each, GET `{participant}/participantSessions` -> `joinTime = min(start)`, `leaveTime = max(end)`, `sessionTimeSec = sum(end - start)`.
3. Resolve emails by matching Meet `displayName` to Firestore `users.displayName` (chunked, 30 at a time) -> { `uid`, `email`, `realName`, `displayName`, `type`: `signedin|anonymous`, `join/leave/session` }.
4. If `classId` + `lectureId` are provided, writes `set(users/{uid}/activities/{lectureId}, { classId, lectureId, meetSession: { joinTime, leaveTime, durationSec, displayName } }, { merge: true })` - this is the auto check-in write, separate from the student's manual `checkedInAt`. On top of `meetSession`, the function also auto-marks attendance from Meet times: `checkedInAt` is always set to the Meet `joinTime` (overwriting any manual check-in). `completedAt` is additionally set to the Meet `leaveTime`, but only for lectures without a required post-test quiz (post-test lectures still require the student to pass the quizzes and tap Mark as Completed manually). `completedAt` is skipped while `leaveTime` is null (meeting still live) and appears on the next Refresh after the meeting ends.

8. Admin Managing Students (and Teachers)

8.1 Per-class enrolment (/[classId]/students -> `EnrolStudents.svelte`)

- Top table: POST `getStudents { classId }` (admin-only; resolves `enroledStudents` with chunked fetch) -> `rama_id`, `name`, `email`, `role`, `year`.
- Bottom: Import from student list, which embeds `ManageStudents`.

8.2 Global student management (/dashboard/students -> ManageStudents.svelte)

- Lists users where role in [student, resident] sorted by createdAt desc in StudentTable.
- Enrol per row -> modal with student details (first/last, role, year y4-y6 / r1-r3, department, electiveStart/End, phone, lineId) + class SearchableSelect -> POST enrolStudents { classId, studentIds: [id] } -> arrayUnion on both users.enroledClasses and classes.enroledStudents.
- StudentTable.svelte columns: Enrol | timestamp | name | email | role | electiveStart/End | hospital | actions; edit opens EditStudentModal; delete calls POST deleteUser { id }.

8.3 Google Sheets import (GoogleSheetsImport.svelte)

- On login auto-calls POST readGoogleSheet {}. The function reads GOOGLE_SHEET_ID / TAB / API_KEY from functions/.env via the Sheets v4 API, selects columns [0,1,2,4,5,7,8,9,11,15,16], drops empty rows, and sorts descending by DD/MM/YYYY HH:mm:ss timestamp.
- Frontend maps Thai status values (นักศึกษาแพทย์ -> student, แพทย์ประจำบ้าน -> resident, อาจารย์ -> teacher), parses elective DD/MM/YYYY, and offers per-row Enrol -> modal to correct role / year / electiveStart / End / classId -> POST enrolStudents { classId, studentData: [...] } variant.

8.4 Teacher management (/dashboard/teachers -> ManageTeachers.svelte)

- Table of users where role == teacher + directory search (POST getPeopleDirectory) -> confirm modal -> POST inviteTeacher { email, name } (creates an invited signedUp: false user; handles already in use).
- Edit/delete mirrors student management. A connect_google_meet response shows a reconnect card calling meetOAuthUrl.

8.5 Authorization summary

- verifyAdmin guards: enrolStudents, getStudents, readGoogleSheet, deleteUser, deleteClass.
- verifyStaff (teacher + admin) guards: Meet create/participants/status/URL/directory endpoints.

9. Data Model Reference

users/{uid}

authId, email, emailLower, name/displayName, photoURL,
gender, role, year, hospital, department?, phone, lineId?,
electiveStart, electiveEnd,
assessmentFormUrl/Path/Name,
enroledClasses[], signedUp, signedUpAt, createdAt

users/{uid}/activities/{lectureId}

```

classId, lectureId, checkedInAt, completedAt,
meetSession { joinTime, leaveTime, durationSec, displayName }

classes/{classId}
  name, code, classStart, classEnd, enroledStudents[], createdAt

classes/{classId}/lectures/{lectureId}
  title, startTime, endTime,
  materials [{ id, type, title, value, requiredPostTest?, meetingHost? }],
  materialsOrder[], sessionData?

classes/{classId}/assignments/{assignmentId}
  title, instructions, opensAt, dueDate,
  requiredAttachments [{ id, instruction }],
  assignedStudentIds[], createdAt

classes/{classId}/assignments/{assignmentId}/submissions/{docId}
  studentUserId, attachments [{ name, filePath, attachmentId, uploadedAt }],
  submittedAt, updatedAt, requirementsMet?, missingAttachmentIds?

quizzes/{quizId}
  title, questions[], passingScore, shuffleQuestions?

quizAttempts/{attemptId}
  quizId, lectureId, authId, score, totalPoints, passed,
  answers [{ questionId, answer, correct }], completedAt

meetOAuth/{uid} { refreshToken }
courseTemplates/{id} { ..., lectureCount }

```

10. Appendix: Routes and Cloud Functions

Key routes:

```

/ -> student login      /signup -> student signup    /login -> admin login
/classes -> class list   /classes/[id] -> class detail (lectures + assignments)
/dashboard -> all classes /dashboard/[classId] -> class editor
/dashboard/[classId]/students | /attendance | /assessment
/dashboard/[classId]/assignments/[assignmentId] (+ /submissions)
/dashboard/[classId]/[lectureId] -> lecture detail
/dashboard/students | /teachers | /templates/[templateId]
/quiz | /quiz/new | /quiz/[id] | /quiz/[id]/edit | /quiz/[id]/take
/quiz/[id]/preview | /quiz/[id]/results (+ /[attemptId])
/settings (+ Google Meet connect)

```

Cloud Functions (base src/lib/functionsUrl.ts):

```

signUpGoogle, googleSignIn, enrolStudents, getStudents, inviteTeacher, deleteUser,

```

**deleteClass, updateUserRole, createUsers, submitQuiz, submitAssignment, createMeetSpace,
getMeetParticipants, getMeetInvitees, getPeopleDirectory, meetOAuthUrl, meetOAuthStatus,
readGoogleSheet, getVideoUploadUrl, getVideoPlaybackUrl, deleteVideo, uploadMaterial**